Online Image Description System

CS3201 2026 Programming Assignment

1 Members and environment

NGUYEN, Phat Dat - 59911219
dnguyen5-c@my.cityu.edu.hk

LE, Win - 59271285
winle2-c@my.cityu.edu.hk

Operating system: Windows, using PowerShell.
Server and client: Python 3.11.15.
Client libraries: Python standard library only.
Setup tool: uv 0.10.9. The server uses the dependencies supplied with the assignment.

2 Setup and execution

We used the Windows setup script from the assignment. Keep the server, setup files, requirements, test_images and models together in the assignment folder. Since both programs are in Python, there is no compilation step. PowerShell blocked our first setup attempt, so we used this command to run the script with a temporary Bypass setting:

```
cd "D:\image-description-assignment\image-description-assignment"
powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "cmd.exe /d /c .\setup_env.bat"
```

After Setup complete, we checked Python and the AI component:

```
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe ImageDescriptionServer.py --check-ai .\test_images\cat.jpg
```

The AI check returned a description of the cat. We then started the real AI server in one PowerShell window and the client in another:

```
.\.venv\Scripts\python.exe ImageDescriptionServer.py
.\.venv\Scripts\python.exe ImageDescriptionClient.py 127.0.0.1 18333
```

We took the screenshots before adding our student IDs to the filenames. The submitted files are ImageDescriptionServer_59911219_59271285.py and ImageDescriptionClient_59911219_59271285.py. To run these files from the assignment folder, use:

```
.\.venv\Scripts\python.exe ImageDescriptionServer_59911219_59271285.py
.\.venv\Scripts\python.exe ImageDescriptionClient_59911219_59271285.py 127.0.0.1 18333
```

Wait for READY before starting the client. Enter each command and each required value on a separate line. QUIT exits the client. Ctrl+C stops the server.

3 Implementation

On the server side, we filled in the five functions in Part 1. We left recv_exact and Parts 2, 3 and 4 as provided in the template.

In create_listener, we use AF_INET and SOCK_STREAM for IPv4 TCP, then bind the socket and call listen(). If this fails, we close the socket before raising the error. accept_client simply returns the connection and address from accept(). For close_client, we call close() on the connected socket.

The main part of receive_message is making sure we get enough bytes. One recv() call may return only part of a message, so we use recv_exact to read the 10-byte header and then the data. The header must contain only digits, and the data length must be from 1 to 5,242,880 bytes. We return None if the client disconnects before a new message starts. If it disconnects halfway through, we raise a connection error.

For send_message, we check the data length and put it in a 10-digit ASCII header, adding zeros at the beginning. We use sendall() to send the header and data completely.

The client keeps the same TCP connection while the user enters commands. It shows client: for input and server: for each line of the reply. For LIST and DESCRIBE, we receive the whole reply first and then print its lines.

We encode text in UTF-8 before counting its bytes. Commands cannot exceed 1,024 bytes. For UPLOAD, we send the command and filename first, then the image in a second message. We do not wait for a reply between them. For DOWNLOAD, we read the text reply first and only receive an image message when the reply starts with OK and a filename.

Before uploading, we check that the local file exists, can be read and is no larger than 5 MiB. We also reject empty files. Only the filename is sent, without the local path. The server checks whether the data is a valid image. When saving a download, we open the destination with xb so it cannot overwrite an existing file. The Saved to message is printed after writing and closing the file.

If the server rejects a command, the client prints its reply and lets the user try again. If a download cannot be saved, the next command still works because we have already received all the image bytes. EOF, Ctrl+C and connection errors end the client and close its socket.

4 Results from our own runs

We tested all six commands with the real AI model on the same Windows computer, using 127.0.0.1 and port 18333. The following screenshots show our terminal output.

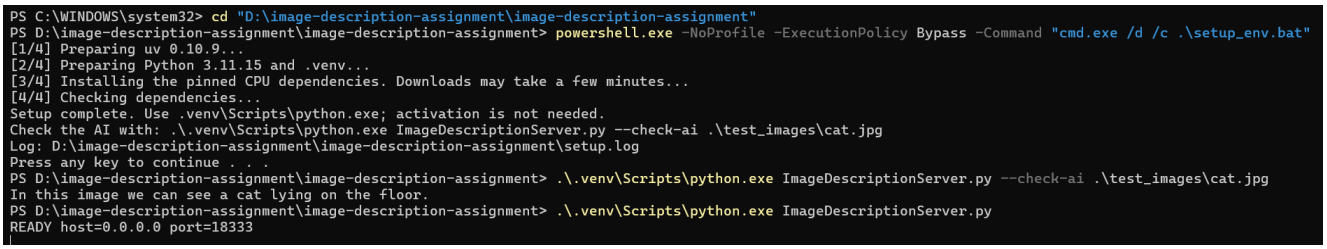


Figure 1. Setup completed, the AI check returned a caption, and the server started on port 18333.

Successful commands and file transfer

```
PS C:\WINDOWS\system32> cd "D:\image-description-assignment\image-description-assignment"
PS D:\image-description-assignment\image-description-assignment> .\.venv\Scripts\python.exe ImageDescriptionClient.py 127.0.0.1 18333
client: LIST
server: OK
server: No images
client: UPLOAD
client: .\test_images\cat.jpg
server: OK 0000
client: DESCRIBE
client: 0000
server: OK
server: In this image we can see a cat lying on the floor.
client: LIST
server: OK
server: IMAGE ID: 0000 | FILE: cat.jpg | TIME: 14:51:15 | DESCRIPTION: yes
client: DOWNLOAD
client: 0000
client: cat_download.jpg
server: OK cat.jpg
client: Saved to cat_download.jpg
client: REMOVE
client: 0000
server: OK
client: LIST
server: OK
server: No images
client: QUIT
server: OK
PS D:\image-description-assignment\image-description-assignment> Get-FileHash .\test_images\cat.jpg, .\cat_download.jpg

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          30C8AEE2B36283E9E980779B66DDB584907754AC888CE177FBD093DDF7FF1476       D:\image-description-assignme...
SHA256          30C8AEE2B36283E9E980779B66DDB584907754AC888CE177FBD093DDF7FF1476       D:\image-description-assignme...
```

Figure 2. A full client session with the real AI model and the SHA-256 check.

The server assigned ID 0000 to cat.jpg. DESCRIBE returned "In this image we can see a cat lying on the floor." The next LIST showed DESCRIPTION: yes. DOWNLOAD saved cat_download.jpg. After REMOVE, LIST returned No images, and QUIT returned OK.

We compared the original and downloaded files with:

```
Get-FileHash .\test_images\cat.jpg, .\cat_download.jpg
```

Both SHA-256 hashes were identical:

```
30C8AEE2B36283E9E980779B66DDB584907754AC888CE177FBD093DDF7FF1476
```

The matching hashes show that the download kept the original file bytes. In Figure 3, another upload has DESCRIPTION: no because we had not requested its description yet.

Error handling

```
PS D:\image-description-assignment\image-description-assignment> .\.venv\Scripts\python.exe ImageDescriptionClient.py 127.0.0.1 18333
client: UNKNOWN
server: ERROR - Command not understood
client: UPLOAD
client: .\missing.jpg
client: ERROR - File not found
client: DESCRIBE
client: 9999
server: ERROR - Invalid ID
client: LIST
server: OK
server: No images
client: UPLOAD
client: .\test_images\cat.jpg
server: OK 0001
client: DOWNLOAD
client: 0001
client: cat_download.jpg
server: OK cat.jpg
client: ERROR - Output file already exists
client: LIST
server: OK
server: IMAGE ID: 0001 | FILE: cat.jpg | TIME: 14:53:32 | DESCRIPTION: no
client: QUIT
server: OK
PS D:\image-description-assignment\image-description-assignment> Set-Content .\invalid.jpg "not an image"
PS D:\image-description-assignment\image-description-assignment> .\.venv\Scripts\python.exe ImageDescriptionClient.py 127.0.0.1 18333
client: UPLOAD
client: .\invalid.jpg
server: ERROR - Invalid image
client: LIST
server: OK
server: IMAGE ID: 0001 | FILE: cat.jpg | TIME: 14:53:32 | DESCRIPTION: no
client: QUIT
server: OK
PS D:\image-description-assignment\image-description-assignment>
```

Figure 3. Unknown command, missing file, invalid ID, existing download destination and invalid image. LIST still works after the errors.

UNKNOWN returned Command not understood. The missing local file was rejected before uploading. DESCRIBE 9999 returned Invalid ID. Uploading a text file named invalid.jpg returned Invalid image.

We also tried downloading to cat_download.jpg again. The client reported Output file already exists, then still accepted LIST and QUIT. The new upload received ID 0001, so the server did not reuse the deleted ID 0000.

Multiple clients



Figure 4. The first client can list the shared image 0001 and then quit.



Figure 5. The second client sees the same image and can still use LIST after the first client quits.

We opened two clients while keeping the same server running. Both could see image 0001. We then quit the first client and ran LIST again in the second one. It still worked. This checked that the clients share the image list and that quitting one client does not close the other connection.

5 Limitations

We tested the real AI server and clients on one computer. We have not tested them between two computers. After leaving a client idle for a while, we got a connection error. We reconnected and could continue testing. The supplied server has a 300-second I/O timeout by default, though we did not confirm the cause of that error.

We kept the limits from the assignment: JPEG or PNG files, at most 5 MiB and 20 million pixels each, and storage for 20 images or 50 MiB. Restarting the server clears the images, cached descriptions and IDs.